

Reviewer Pack — Minimal Order of Formal Power Series and Frege Proof Length ...

Entry bb305220144e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 15:19:12 UTC

Minimal Order of Formal Power Series and Frege Proof Length Correlation

Entry ID: bb305220144e **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 15:19:12 UTC

1. Conjecture

Field A (mathematical branch): Formal Power Series Theory **Field B** (complexity object): Frege Proof Complexity

Statement:

For every CNF φ with n variables, the minimal order of a formal power series representation of φ 's truth table is linearly correlated with its Frege proof length, such that $\text{min_order}(\varphi) = \Theta(f(n))$ where $f(n) = n^{(\log(n)/\log(\log(n)))}$

Rationale (proposer's reasoning):

Formal power series offer a rich structure for encoding computational complexity, and their minimal order could potentially capture the complexity of reasoning about the truth table of a CNF. This correlation might reveal underlying complexities in proof construction that are not immediately apparent.

Taxonomy category: Formal Power Series Theory × Frege Proof Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1bbb53f57bed69b6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

A conjecture is supported if, for at least 80% of the CNFs tested, the Pearson correlation coefficient between $\text{min_order}(\varphi)$ and $f(n)$ exceeds 0.7 with a p-value less than 0.05 across all seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - formal power series AND minimal order IN Frege proof complexity - Frege proof length AND linear correlation WITH formal power series representation - CNF truth table AND order of formal power series IN $\Theta(n^{(\log(n)/\log(\log(n)))})$

Top relevant hits considered: - [http://arxiv.org/abs/2501.05375v2] Some factorization results for formal power series - [http://arxiv.org/abs/1203.2236v1] On Quotients of Formal Power Series - [http://arxiv.org/abs/1705.08630v2] Geometric location of periodic points of 2-ramified power series - [http://arxiv.org/abs/1905.08936v2] Almost all permutations power to a cycle of prime length - [http://arxiv.org/abs/2111.07483v2] Tradeoffs for small-depth Frege proofs - [http://arxiv.org/abs/2401.15683v3] On Small-depth Frege Proofs for PHP - [http://arxiv.org/abs/2508.18079v3] Open Data from LIGO, Virgo, and KAGRA through the First Part of the Fourth Observing Run - [http://arxiv.org/abs/2410.16565v2] Search for gravitational waves emitted from SN 2023ixf

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.3s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2 ** n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(abs(x) != abs(y) for x, y in combinations(clause, 2)):
                clauses.append(clause)
        return clauses

    def min_order(cnf):
        n = len(cnf[0])
        order = 1
        while True:
            valid = True
            for clause in cnf:
                if any(abs(x) == abs(y) for x, y in combinations(clause, 2)):
                    continue
                if all(abs(coeff) < 1e-6 for coeff in [sum([x**i for x in clause]) for i in range(
```

```

        valid = False
        break
    if valid:
        return order
    order += 1

def frege_proof_length(cnf):
    # Simplified DPLL solver to estimate proof length
    stack = []
    assignment = {}
    def dpll():
        if not cnf:
            return True, len(stack)
        literal = next((x for x in range(1, n + 1) if x not in assignment and -x not in assignment))
        if literal is None:
            return False, 0
        assignment[literal] = True
        stack.append(literal)
        for clause in cnf:
            if all(x not in assignment or assignment[x] != (x > 0) for x in clause):
                continue
            if any(-x not in assignment or assignment[-x] != (x < 0) for x in clause):
                return False, 0
        result, length = dpll()
        if result:
            return True, length
        del assignment[literal]
        stack.pop()
        assignment[-literal] = True
        stack.append(-literal)
        for clause in cnf:
            if all(x not in assignment or assignment[x] != (x > 0) for x in clause):
                continue
            if any(-x not in assignment or assignment[-x] != (x < 0) for x in clause):
                return False, 0
        result, length = dpll()
        if result:
            return True, length
        del assignment[-literal]
        stack.pop()
        return False, 0
    return dpll()[1]

n_values = [5, 10, 15, 20, 30, 40]
min_orders = []
proof_lengths = []

for n in n_values:
    cnf = generate_cnf(n)
    min_order_value = min_order(cnf)
    proof_length_value = frege_proof_length(cnf)
    min_orders.append(min_order_value)
    proof_lengths.append(proof_length_value)

if len(min_orders) < 30 or len(proof_lengths) < 30:
    return {

```

```

        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": len(min_orders),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "not_enough_instances"
    }

mean_min_order = sum(min_orders) / len(min_orders)
mean_proof_length = sum(proof_lengths) / len(proof_lengths)
correlation_coefficient = 0.0
for i in range(len(min_orders)):
    correlation_coefficient += (min_orders[i] - mean_min_order) * (proof_lengths[i] - mean_proof_length)
correlation_coefficient /= len(min_orders) * math.sqrt(sum((x - mean_min_order)**2 for x in min_orders))

return {
    "metric_name": "Pearson correlation coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(min_orders),
    "n_max": max(n_values),
    "conjecture_holds": abs(correlation_coefficient) > 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        for r in results:
            if not r["conjecture_holds"]:
                counterexample = f"n={r['instances_tested']}, min_order={min(r['metric_name'], key=lambda x: x['min_order'])}"
                print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={seed}")
                break

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it was unable to complete the required calculations to determine the Pearson correlation coefficient and p-value. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14652
2	propose	ollama_remote	glm4:latest	0	0	12903
3	propose	ollama_remote	glm4:latest	0	0	12453
4	propose	ollama_remote	glm4:latest	0	0	12712
5	preregistration	ollama_remote	glm4:latest	0	0	30363
6	novelty	ollama_remote	glm4:latest	0	0	11984
7	novelty	ollama_remote	glm4:latest	0	0	11316
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16481
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24631
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17707
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26681
12	judge	ollama_remote	glm4:latest	0	0	8767

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 200650 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in research/audit/bb305220144e.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/bb305220144e.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/bb305220144e.tar.gz (if generated)